

Bayesian Knowledge Tracing (BKT)

How the platform estimates what each student knows — explained from your code

Intelligent Adaptive Learning Platform · Yassine Ben Nessib · EspritTech — explained from the real source code

In one sentence — BKT keeps a probability “how likely is it that this student masters this concept?”, and updates it after every answer with Bayes’ rule, correcting for lucky guesses and careless slips. All of it lives in `bkt_service.py`.

1 The idea in one minute

We only see quiz answers, and answers are noisy: a student can be right by **luck** (a guess) or wrong by a careless **slip**. A plain average is fooled by this. BKT instead keeps a **probability** of mastery and updates it carefully after each answer. It replaces the old rule that used to be

$$\text{mastery} = 0.6 * \text{old} + 0.4 * \text{new}.$$

2 Where it lives in the code

`app/services/bkt_service.py`

The model itself: parameters + the Bayes update.

`app/data/bkt_params.json`

Optional fitted parameters per concept (else defaults).

`scripts/fit_bkt.py`

Learns the 4 parameters from real logs (EM).

Mastery model (DB)

Stores the probability per (student, concept).

3 The four numbers the model uses

BKT_SERVICE.PY — THE DEFAULT PARAMETERS

```
DEFAULT_PARAMS = {  
    "p_L0": 0.20,    # prior knowledge (before any practice)  
    "p_T": 0.15,     # learning rate (chance to learn between two questions)  
    "p_S": 0.10,     # slip (knows, but answers wrong)  
    "p_G": 0.25,     # guess (does not know, but answers right -> 1/4 on a 4-choice MCQ)  
}
```

What this does, step by step:

- A Python `dict` (a labelled box of values) holds the four BKT parameters.
- **p_L0** = how likely the student already knows the concept at the start.
- **p_T** = how likely they learn it between two questions.
- **p_S** (slip) and **p_G** (guess) model the noise. Guess is 0.25 because a 4-choice question can be answered right 1 time in 4 by luck.
- `get_params(concept_id)` returns these defaults, OR fitted values from `bkt_params.json` if they exist for that concept.

4 The heart: one update step (Bayes + learning)

This is the most important function. Each answer triggers **two steps**: (1) correct our belief using the answer, then (2) allow for the fact the student may have just learned.

BKT_SERVICE.PY — POSTERIOR_AFTER_OBS() (THE CORE)

```
def posterior_after_obs(p_known, correct, params):
    s, g = params["p_S"], params["p_G"]
    p = _clamp(p_known)          # keep p strictly between 0 and 1
    if correct:                  # Bayes update if the answer is CORRECT
        num = p * (1.0 - s)
        den = p * (1.0 - s) + (1.0 - p) * g
    else:                        # Bayes update if the answer is WRONG
        num = p * s
        den = p * s + (1.0 - p) * (1.0 - g)
    p_post = (num / den) if den > 1e-9 else p
    # Learning transition: maybe the student just learnt it.
    p_next = p_post + (1.0 - p_post) * params["p_T"]
    return _clamp(p_next)
```

What this does, step by step:

- **Inputs:** `p_known` = our current belief (a number 0-1), `correct` = True/False, and the 4 parameters.
- `_clamp(p)` just forces `p` to stay between 0 and 1 (never exactly 0 or 1), to avoid dividing by zero.
- **If the answer is correct:** we apply Bayes. `num` = the chance to be right *because you know it* (`p` × not-slip). `den` = total chance to be right (know & not slip + don't-know & guess). The fraction `num/den` is the new belief.
- **If wrong:** same idea reversed — being wrong while knowing (slip) vs being wrong while not knowing.
- **Last two lines** = the learning step: even after the answer, the student might have just learned, so we push the belief up by `p_T`.
- It returns `p_next` = the updated probability of mastery. This single number is what the platform saves.

THE SAME THING AS A FORMULA (CORRECT ANSWER)

$P(\text{known} \mid \text{correct}) = [p \cdot (1 - s)] / [p \cdot (1 - s) + (1 - p) \cdot g]$ then $P(\text{next}) = P(\text{known} \mid \text{correct}) + (1 - P(\text{known} \mid \text{correct})) \cdot p_T$

5 Running it over a whole quiz

BKT_SERVICE.PY — UPDATE_MASTERY() (LOOPS OVER ANSWERS)

```
def update_mastery(concept_id, prior_p, observations):
    params = get_params(concept_id)
    p = params["p_L0"] if prior_p is None else _clamp(prior_p)
    for correct in observations:          # one update per answer
        p = posterior_after_obs(p, bool(correct), params)
    return p
```

What this does, step by step:

- `observations` is a list of True/False (the student's answers on this concept).
- We start from the saved belief `prior_p`, or from `p_L0` if the student is new.
- The `for` loop applies the update once per answer, feeding the result back in — the belief moves step by step.
- It returns the final mastery probability after the whole quiz.

6 Predicting the next answer

BKT_SERVICE.PY — PREDICT_CORRECT()

```
def predict_correct(concept_id, p_known):
    params = get_params(concept_id)
    p = _clamp(p_known)
    return p * (1.0 - params["p_S"]) + (1.0 - p) * params["p_G"]
```

What this does, step by step:

- Given the current mastery `p_known`, this estimates the chance the student answers the next question correctly.
- It mixes two cases: knows & doesn't slip, OR doesn't know & guesses right. The adaptive engine uses this to pick a question that is neither too easy nor too hard.

7 How the data flows at runtime

- 1 **Student answers a quiz** — each question is correct/incorrect on a concept.
- 2 **update_mastery() runs** — starting from the saved probability, it applies one Bayes+learning step per answer.
- 3 **Mastery is saved** — the new probability is stored per (student, concept) in the database.
- 4 **Adaptive engine reacts** — low mastery → review/easier content; high mastery → move forward.

8 Worked example (real numbers)

One correct answer

Start $p=0.30$, with $G=0.25$, $S=0.10$, $T=0.15$.

Correct $\rightarrow \text{num} = 0.30 \times 0.90 = 0.27$; $\text{den} = 0.27 + 0.70 \times 0.25 = 0.445 \rightarrow p_{\text{post}} = 0.27/0.445 \approx \mathbf{0.607}$.

Learning $\rightarrow p_{\text{next}} = 0.607 + (1-0.607) \times 0.15 \approx \mathbf{0.666}$.

So one correct answer moved the belief from 30% to ~67% — not 100%, because it might be a guess.

9 Strengths & limits

Strengths

- Clear, interpretable probability
- Only 4 parameters — little data needed
- Handles guess & slip correctly
- A proven standard in EdTech

Limits

- Assumes no forgetting
- Knowledge is binary (know / not)
- Concepts treated independently
- Parameters must be set or fitted

Honest note — The 4 parameters use sensible defaults and can be **fitted** from real logs with `scripts/fit_bkt.py` (an EM algorithm). Until you have enough real data, the defaults are used — say this honestly if asked.

10 Q&A

DEFENSE / INTERVIEW QUESTIONS (INCLUDING “WHAT DOES THIS BLOCK DO?”)

Q What does the function `posterior_after_obs` do?

A It performs one BKT step on a single answer: a Bayes update (correcting for slip/guess) followed by the learning transition. It returns the new mastery probability.

Q Why two branches (if correct / else)?

A Because Bayes' rule is different for a right answer and a wrong answer. A right answer raises the belief (unless it was a guess); a wrong answer lowers it (unless it was a slip).

Q What is `_clamp` and why?

A A tiny helper that keeps the probability strictly between 0 and 1, so we never divide by zero or get stuck at a value that can never change.

Q Why is `p_G` 0.25?

A Our quizzes are 4-choice, so a pure guess is right about 1 time in 4.

Q How would you improve BKT?

A Fit the parameters per concept from real logs (fit_bkt.py), add forgetting, or move to Deep Knowledge Tracing (a neural model) when enough data exists.

11 Key terms

dict — A Python labelled box: name → value (here the 4 parameters).

Prior — Belief before the new answer (p_{L0} or the saved value).

Posterior — Belief after the answer (the Bayes result).

Slip / Guess — The noise: wrong-while-knowing / right-while-not-knowing.

Transition (p_T) — The chance to have learned between two questions.

Clamp — Force a number to stay inside a range.